Model selection and evaluation workshop

1. What You'll Learn in This Section

In this lesson, you'll learn to:

> Distinguish regression problems from classification problems by looking at the output variable.
> Explain why regression and classification models can never be validly compared to each other.
> Compute and interpret regression metrics (MSE, RMSE, R^2) and classification metrics (accuracy, precision, recall, F1) from a confusion matrix.
> Apply the scikit-learn workflow — including `StandardScaler` and stratified sampling — to compare two candidate models on the same problem type.

2. Detailed Explanation

Regression vs. classification: it all starts with the output variable

Correct model selection starts before any code is written: it depends entirely on recognizing what type of output variable the problem statement demands. Imagine a delivery company asking two different questions about the very same shipments:

1. How many minutes will this delivery take?
2. Will this delivery be late or on time?

Both questions come from the same company and the same underlying data, but they demand different types of models. The first question expects a numeric answer that can take any value on a continuous scale — a delivery could take 1 minute or 1,000 minutes, even down to fractions like 30 minutes 30 seconds. This makes it a regression problem: the output variable is continuous.

The second question expects exactly one of two outcomes — late or on time. This is a classification problem: the output variable is discrete (binary here).

The type of output variable is what decides which family of algorithm applies. A continuous output calls for a regression algorithm such as linear regression; a discrete or categorical output calls for a classification algorithm such as logistic regression. Both families still follow the same general workflow — import or create the dataset, choose the algorithm, train it, and predict. In scikit-learn, both use the identical interface: a `.fit()` call to train and a `.predict()` call to generate outputs, regardless of whether the underlying algorithm is a regressor or a classifier.

Watch out for: the people requesting a model (say, a business team) often have no machine learning background. They may not realize they are actually asking for two structurally different solutions, so it falls on whoever builds the models to recognize the output-variable type before writing any code.

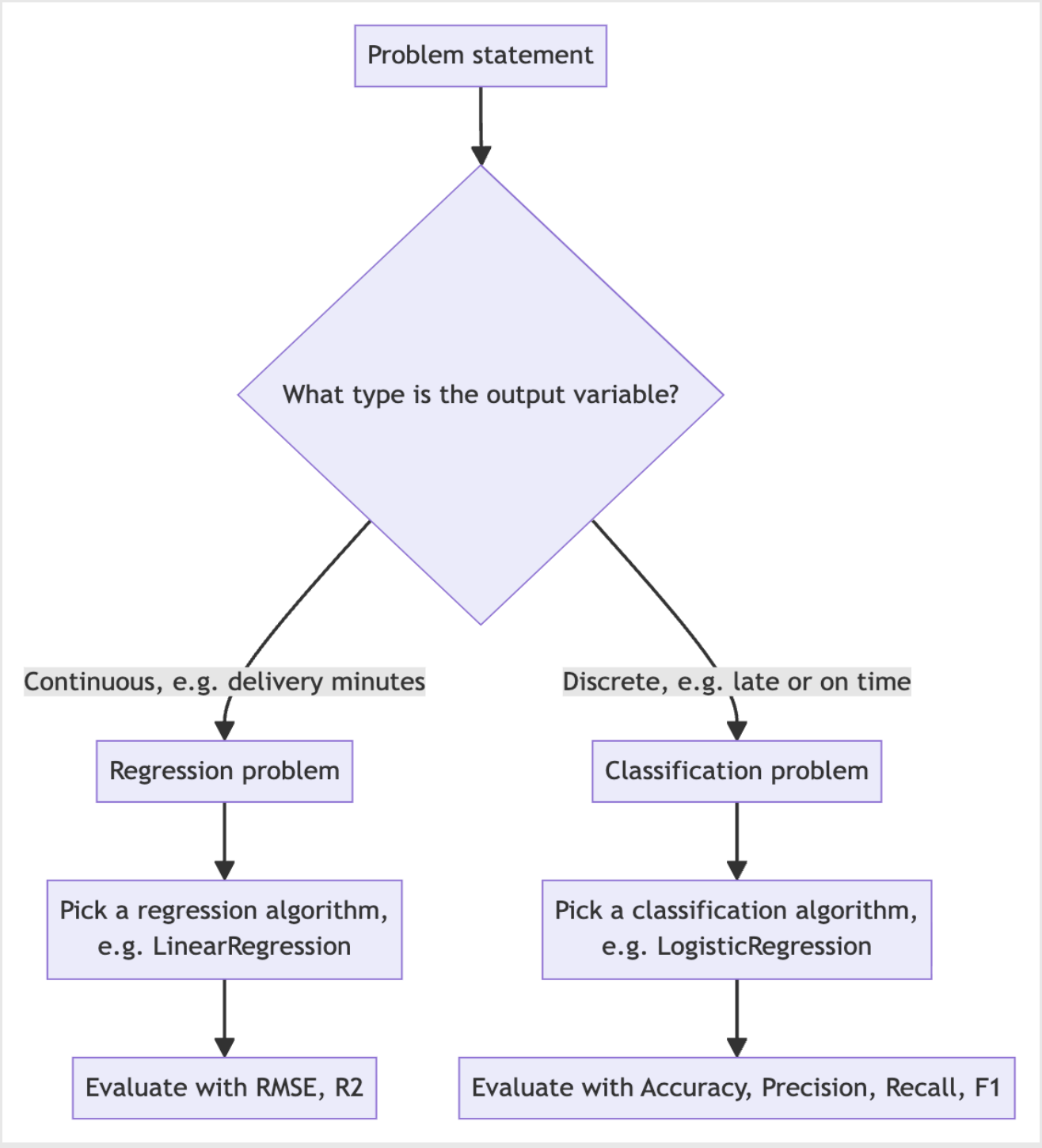
Why regression and classification models cannot be compared to each other

Two distinct questions can be asked about a pair of models built for different problem types. First: is each individual model good? That's answered independently — a regression model is judged against other regression models solving the same problem, and a classification model against other classification models solving its problem. Second: can the two models be compared against each other? That question is baseless. A regression model cannot be meaningfully compared to a classification model, because they are trained for different kinds of problems and produce fundamentally different output — one continuous, one binary.

Attempting the comparison is like asking how nutritious an orange is compared to an apple. An orange is rich in vitamin C, an apple is rich in vitamin A, and there's no shared dimension on which to declare a winner. A regression model and a classification model share no common evaluation ground; each can only be validly compared against other models solving the *same* type of problem. Model selection is decided by the shape of the problem, not by preference.

The two problem types also fail differently, which is exactly why they need different evaluation metrics:

> In regression, any value on the number line is a technically valid prediction. If the true delivery time is 36 minutes and the model predicts 38, 38.4, 40, or 28 minutes, none of these is "wrong" in a binary sense — they simply carry more or less error.
> In classification, a prediction either matches the correct class or it doesn't. If the correct label was "on time" and the model predicted "late," that single instance is a complete failure — a 100% miss for that case — and it directly hurts the overall evaluation score.



Evaluation metrics for regression

For regression problems, the residual (the difference between actual and predicted values) is squared — turning any negative differences into positive numbers — before being averaged. Three core metrics come out of this:

Mean Squared Error (MSE): the average of the squared residuals across all data points.

Root Mean Squared Error (RMSE): the square root of MSE, which brings the error back into the same units as the original variable (e.g., minutes), making it easier to interpret.
R-squared (R^2): the proportion of variance in the actual data that is captured by the predicted data. R^2 usually falls between 0 and 1, though a poorly fitting model can push it below 0; a value closer to 1 means the predicted values track the actual values' variance closely.

A small toy dataset of six actual-vs-predicted delivery times (in minutes) illustrates this. Example matched pairs included actual 32 minutes predicted as 30, actual 45 predicted as 48, actual 28 predicted as roughly 30, and actual 52 predicted as 50. Across the six points, the error stayed within about plus or minus three minutes. For this dataset, RMSE ≈ 2.24, matching the observation that errors were within about ±3 minutes. $R^2 \approx 0.934$, meaning roughly 93% of the variance in the actual delivery times was captured by the predicted values — reasonably good, though not perfect.

Evaluation metrics for classification

Classification evaluation is built from a confusion matrix, a table that cross-tabulates actual labels against predicted labels. Before building it, one class must be designated the positive class based on business interest. In the delivery scenario, "late" deliveries are the positive class because the business specifically wants to flag them; "on time" is the negative class.

The four confusion matrix outcomes are:

Outcome	Meaning
True Positive (TP)	actual late, predicted late
True Negative (TN)	actual on time, predicted on time
False Positive (FP)	actual on time, predicted late (a false alarm)
False Negative (FN)	actual late, predicted on time (a missed case)

Using a toy dataset of 20 deliveries: TP = 5, FN = 3, TN = 10, FP = 2. From these four counts:

Accuracy = (TP + TN) / Total = (5 + 10) / 20 = 0.75. Accuracy alone does not reveal how many true positive cases were missed, nor does it separate false negatives from false positives.
Precision = TP / (TP + FP) = 5 / 7 ≈ 0.71. Precision answers: out of everything predicted positive, how much was actually positive?
Recall = TP / (TP + FN) = 5 / 8 = 0.625. Recall answers: out of everything actually positive, how much was correctly caught?
F1 score = 2 × (Precision × Recall) / (Precision + Recall) ≈ 0.667 (66.7%), the harmonic mean of precision and recall.

The gap between accuracy (75%) and F1 (66.7%) is the key lesson here: accuracy alone looked acceptable, but the model's recall was weak — it was missing a meaningful share of true late deliveries. F1, by combining precision and recall, exposes that weakness where raw accuracy hides it.

Why you cannot average metrics across problem types

A natural but flawed instinct is to try to summarize both a regression result and a classification result together — for example, averaging R^2 (0.934) and F1 (0.667) into a single number. That produces (0.934 + 0.667) / 2 ≈ 0.80, which looks like a strong combined score. But this naive average is meaningless and actively hides information. The strong regression R^2 of 0.934 gets diluted for no reason, and the classification model's real weakness — a recall of only 0.625, meaning it misses roughly every third actual late delivery — disappears inside the averaged number. Regression and classification metrics must always be reported and interpreted separately, never merged.

The scikit-learn workflow and score-card functions

Both regression and classification models in scikit-learn follow the same five-step programming pattern:

1. Import or load the dataset.
2. Import the desired algorithm class (e.g., `LinearRegression`, `LogisticRegression`, `DecisionTreeRegressor`, `DecisionTreeClassifier`).
3. Call `.fit(X_train, y_train)` to train.
4. Call `.predict(X_test)` to generate predictions.
5. Evaluate using the metrics appropriate to the problem type.

Regression and classification metric functions can be imported together from the same `sklearn.metrics` module using a single parenthesized import statement, since scikit-learn provides functions for both problem types within that one module. A fixed random seed of 42 was used throughout wherever random splitting or randomized behavior was involved, so results stay reproducible across runs.

```
from sklearn.metrics import (
    root_mean_squared_error, r2_score,
    accuracy_score, precision_score, recall_score, f1_score,
    ConfusionMatrixDisplay, classification_report
)
```

Two separate helper functions were built — one for regression metrics, one for classification metrics — rather than combining them into a single function. The two sets of metrics are never applicable at the same time: whenever R^2 and RMSE apply, accuracy/precision/recall/F1 do not, and vice versa.

```
def regression_score_card(y_true, y_pred):
    rmse = root_mean_squared_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(rmse, r2)
    return rmse, r2

def classification_score_card(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
```

```
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    print(acc, prec, rec, f1)
    return acc, prec, rec, f1
```

Running `regression_score_card` on the toy delivery dataset returned RMSE ≈ 2.24 and R^2 ≈ 0.934, matching the values computed manually. Running `classification_score_card` on the toy delivery-lateness dataset returned accuracy 0.75, precision 0.71, recall 0.625, and F1 0.667 — again matching the manual computation.

For visualizing regression predictions, a scatter plot is built with `plt.subplots()`. A diagonal dashed gray line is plotted between two fixed axis limits and labeled "Perfect prediction," representing the ideal case where actual equals predicted; the actual-vs-predicted points are then scattered over that line, with `xlabel`, `ylabel`, a title, a legend, `tight_layout()`, and `plt.show()`. Points sitting close to the diagonal visually confirm a well-performing model.

For visualizing classification results, `ConfusionMatrixDisplay.from_predictions()` takes the actual and predicted labels directly and builds and renders the confusion matrix internally, without a separately computed matrix beforehand. Typical parameters include `display_labels=['on time', 'late']`, a yellow-to-red color map, `colorbar=False`, and a descriptive title, followed by `tight_layout()` and `plt.show()`.

Applying the workflow to real data: the diabetes dataset (regression)

Two real, publicly available biomedical datasets bundled with scikit-learn demonstrate genuine model selection in practice. This time the comparison behind the selection decision is valid, because within each dataset two candidate models are trained and tested on identical data for the identical problem type — the better-performing one is selected using the metrics appropriate for that problem type.

The diabetes dataset contains 442 real patient records, each with 10 numeric measurements (features), and a target value representing diabetes disease progression score — a continuous number (e.g., 100, 200, 300). This is a regression problem.

```
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

tree_reg = DecisionTreeRegressor(max_depth=4, random_state=42)
tree_reg.fit(X_train, y_train)
```

Two regression models were compared: linear regression and a decision tree regressor (`max_depth=4`). A decision tree can be adapted for regression by applying numeric thresholds at each split to decide whether

to branch left or right. Decision trees are, by nature, better suited to classification, but this adaptation still lets them handle regression tasks. Decision trees were historically favored as a "savior" algorithm when a dataset was too small for learning-based algorithms like linear regression, which need enough data to estimate reliable weight parameters.

Model	RMSE	R^2
Linear regression	53	0.4777
Decision tree regression	60	0.324

Linear regression outperformed the decision tree regressor on both metrics — lower RMSE and higher R^2. Neither model performed extremely well (moderate R^2 values), suggesting the dataset would benefit from further refinement, but the comparison itself is valid because both models were trained and tested on identical data. A two-panel plot (`plt.subplots(1, 2, ...)`) placed linear regression's predictions beside the decision tree's, each against a diagonal "perfect prediction" line. Axis limits were derived dynamically from the test target's min minus 20 and max plus 20. A `zip()` loop iterated over the two subplot axes, the two models' predicted arrays, and the two model names together in a single loop. A simple `if` condition then compared the two RMSE values to print a "winner" label.

Applying the workflow to real data: the breast cancer dataset (classification)

The breast cancer dataset contains 569 real diagnostic cases, each labeled either malignant or benign. In the dataset's original encoding, 0 represents malignant and 1 represents benign. Because the goal is a model that reliably flags the more dangerous condition, malignant was deliberately re-mapped to be the positive class (label 1) and benign to the negative class (label 0):

```
cancer = load_breast_cancer()
X = cancer.data
y_cancer = (cancer.target == 0).astype(int)    # malignant -> 1, benign -> 0

X_train, X_test, y_train, y_test = train_test_split(
    X, y_cancer, test_size=0.3, random_state=42, stratify=y_cancer
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_clf = LogisticRegression(max_iter=5000)
log_clf.fit(X_train_scaled, y_train)

tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)    # no scaling needed for a decision tree
```

Two classification models were compared: logistic regression and a decision tree classifier.

Model	Accuracy	Precision	Recall	F1
Logistic regression	0.97	0.98	0.94	0.96
Decision tree classifier	0.90	0.96	0.78	0.862

The dataset is imbalanced (far fewer malignant cases than benign), which is why accuracy alone (a 7-point gap between the two models) understates the real difference between them; the F1 gap (about 10 points) more clearly exposes logistic regression's superiority. Out of 64 actual malignant cases in the test set, logistic regression correctly identified 60 (missing only 4), while the decision tree correctly identified only 50 (missing 14). For benign cases, both models performed similarly well, correctly predicting roughly 105–106 out of the benign cases as benign. A two-panel confusion-matrix visualization placed the logistic regression confusion matrix beside the decision tree's, making the gap in malignant detection visually apparent.

The `classification_report` function prints per-class precision, recall, and F1 in one call by passing the true labels, predicted labels, and class names (e.g., `target_names=['benign', 'malignant']`), giving a class-wise breakdown rather than a single aggregated number per metric. Extending confusion-matrix-based evaluation beyond two classes (multi-class classification) is a topic for deeper treatment later.

Data preprocessing: feature scaling and stratified sampling

Real-world features often live on very different numeric scales. Order quantity might range from roughly 5 to 20 items, while distance might range from roughly 1 to 25 kilometers. A categorical weather variable, if numerically encoded (rainy = 1, cold = 2, sunny = 0), sits on yet another scale. Feeding such mismatched scales directly into a learning algorithm can slow down training.

StandardScaler, available in scikit-learn's preprocessing module, transforms numeric features into a standard normal distribution — mean 0 and standard deviation 1 — which speeds up training for learning-based algorithms. Scaling only applies to numeric features; categorical data is already considered "normalized" in this sense and is left untouched. Calling `.fit_transform()` does two things: first the scaler *fits* — determining the mean and standard deviation needed to normalize each numeric feature — and second it *transforms* the data using those statistics.

Watch out for: the scaler must be fit only on the training data, never on the test data. The mean and standard deviation are computed exclusively from the training set; the test set is only transformed (not fit) using those same training-derived statistics. Fitting on test data would leak information from the test set into preprocessing, since scaling is itself considered part of the training process. Decision trees, by contrast, do not require scaling at all. A decision tree simply applies a sequence of if-else threshold checks at each split, so the relative scale of features doesn't affect its logic or speed.

Stratified sampling ensures that the proportion of each class is preserved across both the train and test splits, using the `stratify=` parameter of `train_test_split` (set to the target variable). For example, if malignant cases make up a certain percentage of the full dataset, that same percentage should appear in both the 70% training portion and the 30% testing portion. This avoids an uneven, purely random split that

over- or under-represents one class in either portion. This concept generalizes to any number of classes, not just two.

Learning-based algorithms vs. rule-based algorithms

Learning-based algorithms — including linear regression, logistic regression, support vector machines (SVM), neural networks, and deep neural networks — produce a mathematical function of the form `w0*x0 + w1*x1 + w2*x2 + ...`. The weights are learned from the actual input-output pairs in the training data through an iterative optimization process. For the breast cancer example, logistic regression ran for 5,000 iterations, progressively refining its weights each time; after a certain point, additional iterations yield diminishing returns as the model's results become stagnant.

Rule-based algorithms — including decision trees and random forests — are not learning-based in the same sense. They compute a simple score (such as a Gini index) at each step to decide where to split the data. Once a split decision is made at a given level, it is not revisited or continuously refined. This makes them structurally simpler and less powerful in principle than learning-based algorithms, but also far less data-hungry and computationally lighter. That trade-off made them historically useful when hardware couldn't support training learning-based models, and it still makes them useful today when the available dataset is small.

Learning-based algorithms can adjust their weights to pay more attention to a minority but important class, such as malignant tumor cases. Because of this, they tend to outperform rule-based algorithms on imbalanced datasets — demonstrated by logistic regression's stronger recall and F1 score on the breast cancer dataset compared to the decision tree classifier.

3. Key Takeaways

Model selection is decided by the shape of the problem, not by preference.
Look at the output variable first — continuous means regression, discrete/categorical means classification — that single decision determines the entire algorithm family.
Regression and classification models can never be validly compared to each other (like comparing an orange to an apple); each is only comparable to other models of the same problem type.
Regression is scored with MSE, RMSE, and R^2; classification is scored with a confusion matrix (TP, TN, FP, FN) feeding accuracy, precision, recall, and F1.
Never average metrics from the two families together.
Accuracy can hide weaknesses, especially on imbalanced data (like the breast cancer dataset); F1 and per-class recall reveal what accuracy misses.
`StandardScaler` (fit only on training data) speeds up learning-based algorithms, while decision trees need no scaling; `stratify=` in `train_test_split` keeps class proportions consistent across train and test sets.

Mental model: Think of the output variable as a fork in the road at the very start of a project. One path leads to regression tools (RMSE, R^2, a number line of predictions); the other leads to classification tools (a confusion matrix, precision, recall, F1). Once you've picked a path, you only ever compare your model to others walking the same path.